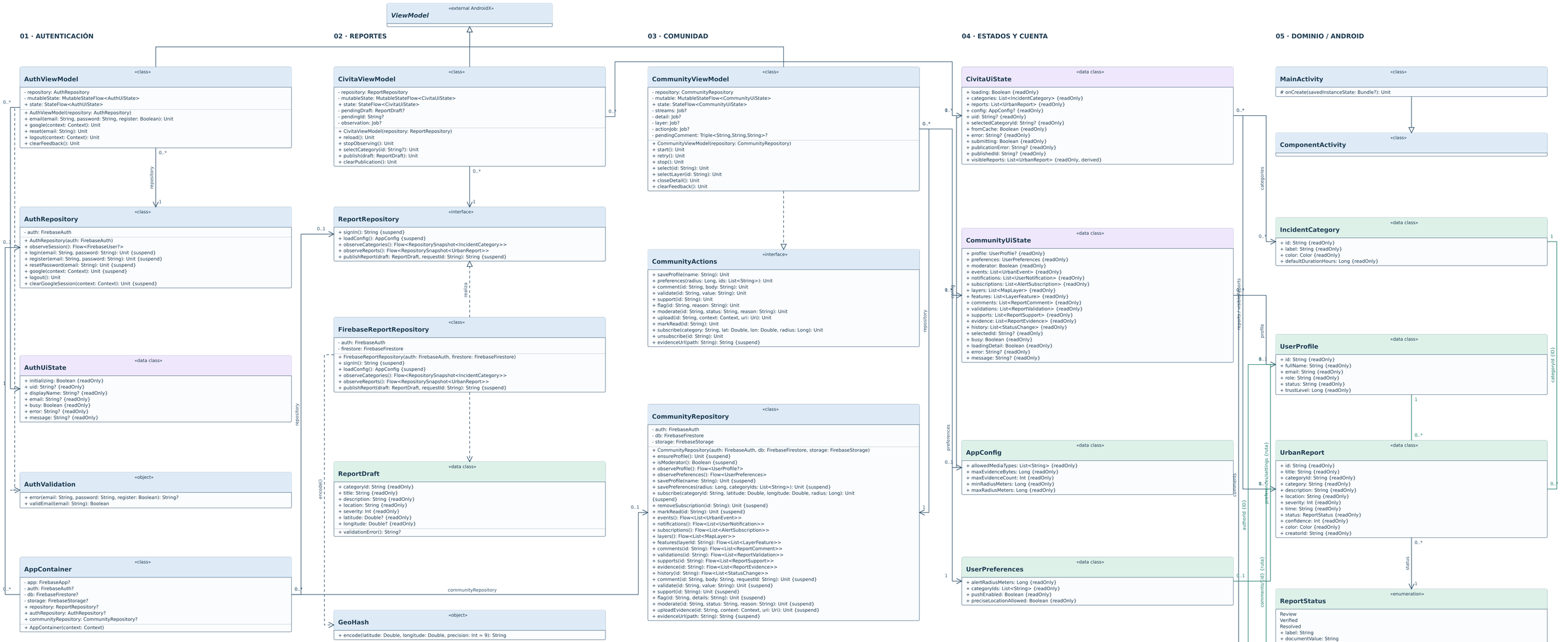
CIVITA · DIAGRAMA DE CLASES · VISTA HORIZONTAL

Cliente Android de despliegue monolítico (:app / una APK) · Diseño interno MVVM por capas + Repository + inyección manual



RELACIONES Y ARQUITECTURA

Triángulo hueco continuo: generalización (ViewModels → ViewModel y MainActivity → ComponentActivity).

Triángulo hueco discontinuo: realización (FirebaseReportRepository → ReportRepository; CommunityViewModel → CommunityActions).

Flecha abierta continua: referencia navegable; state.value es la referencia derivada de StateFlow. Flecha abierta discontinua: dependencia.

Línea verde: asociación lógica por ID o ruta Firestore. 1 = uno; 0..1 = opcional; 0..* = varios. Las dependencias no llevan cardinalidad.

+ público; - privado; # protegido; ? nullable; {readOnly} propiedad val; {suspend} operación coroutine. Entidades Kotlin sin herencia entre sí.

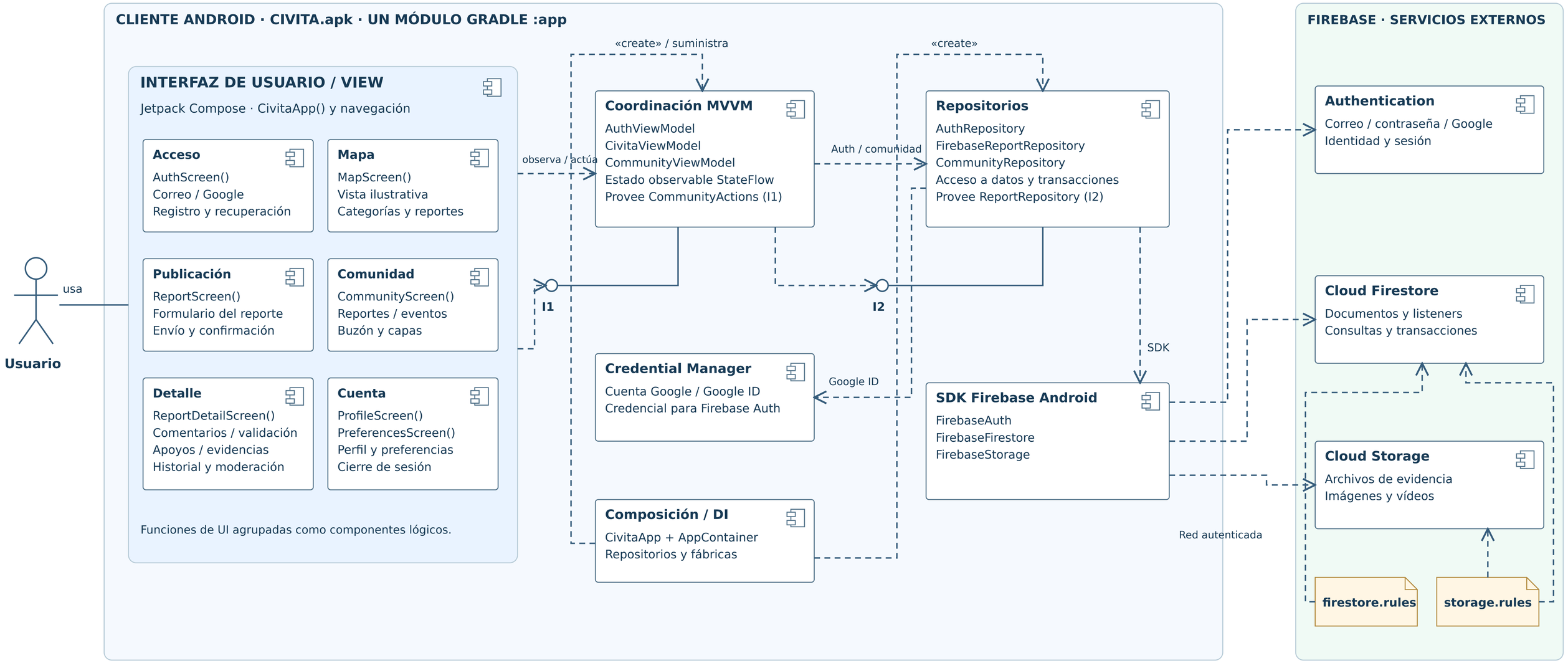
Una APK / módulo :app: monolito en despliegue del cliente. MVVM y Repository describen su organización interna; Firebase es un backend externo BaaS.

Clases principales: los tipos de detalle y fábricas están en el catálogo. No se afirman composición exclusiva ni eliminación automática en cascada.

Fuente: implementación Kotlin CIVITA 1.1.742 · UML 2.5.1 · Una sola página A0 horizontal · PDF y SVG vectoriales ampliables

CIVITA · COMPONENTES · INTERFAZ DE USUARIO Y BACKEND

Diseño interno: MVVM + Repository · Despliegue: cliente monolítico Android (una APK) + Firebase BaaS externo



CÓMO EXPLICAR LA ARQUITECTURA EN LA SUSTENTACIÓN

Despliegue del cliente: monolítico. Todo el código Android se compila en :app y se distribuye en una sola APK.

Diseño interno: MVVM por capas. View = pantallas Compose; ViewModel = coordinación/StateFlow; Model = tipos y repositorios.

Interfaz de usuario: bloque View de la izquierda. Contratos UML: I1 = CommunityActions; I2 = ReportRepository (círculos).

Los componentes dentro de la APK son responsabilidades lógicas; Firebase es un backend administrado externo, accedido mediante sus SDK.

Flecha discontinua abierta: dependencia de uso. Círculo: interfaz proporcionada. Línea continua al actor: interacción con la UI.

Cloud Shell publica reglas/configuración por separado. Storage requiere bucket y permisos; sus reglas consultan autor/contador en Firestore.

Fuente: settings.gradle.kts, CivitaApp.kt, ui/screens/*.kt, ViewModels y repositorios · UML 2.5.1 · Una sola página horizontal.